

```

"""
Wallet screens for the sandbox crypto app (fake
money only).

Put this file in the SAME folder as main.py, then
run:
    uvicorn wallet_app:app --host 0.0.0.0 --port
    8000
"""
from uuid import uuid4

from fastapi import Depends, HTTPException
from fastapi.responses import HTMLResponse
from pydantic import BaseModel, Field

from main import ENV, app, credit, current_user,
db, debit, verified_user

FAKE_PRICE_PENCE = 5_000_000    # pretend 1 BTC =
50,000.00 GBP
SATS_PER_BTC = 100_000_000

class TestMoney(BaseModel):
    amount: int = Field(gt=0, le=10_000_000,
description="pence")

class Trade(BaseModel):
    gbp_pence: int = Field(gt=0, description="GBP
value in pence")

@app.post("/dev/add-test-money", status_code=201)
def add_test_money(body: TestMoney,
user=Depends(verified_user), conn=Depends(db)):

```

```

        # Stands in for a real bank deposit while you
        are in test mode.
        if ENV != "dev":
            raise HTTPException(404, "Not found")
        credit(conn, user["id"], "GBP", body.amount,
            "test_deposit", None)
        return {"status": "ok"}

@app.post("/trade/buy", status_code=201)
def buy_btc(body: Trade,
    user=Depends(verified_user), conn=Depends(db)):
    sats = body.gbp_pence * SATS_PER_BTC //
    FAKE_PRICE_PENCE
    if sats <= 0:
        raise HTTPException(400, "Amount too
small")
    ref = str(uuid4())
    debit(conn, user["id"], "GBP", body.gbp_pence,
        "buy_btc", ref) # fails if not enough GBP
    credit(conn, user["id"], "BTC", sats,
        "buy_btc", ref)
    return {"gbp_pence": body.gbp_pence,
        "btc_sats": sats}

@app.post("/trade/sell", status_code=201)
def sell_btc(body: Trade,
    user=Depends(verified_user), conn=Depends(db)):
    sats = body.gbp_pence * SATS_PER_BTC //
    FAKE_PRICE_PENCE
    if sats <= 0:
        raise HTTPException(400, "Amount too
small")
    ref = str(uuid4())
    debit(conn, user["id"], "BTC", sats,
        "sell_btc", ref) # fails if not enough
    BTC
    credit(conn, user["id"], "GBP", body.gbp_pence,
        "sell_btc", ref)

```

```
    return {"gbp_pence": body.gbp_pence,  
    "btc_sats": sats}
```

```
PAGE = ""<!doctype html>  
<html lang="en">  
<head>  
<meta charset="utf-8">  
<meta name="viewport" content="width=device-width,  
initial-scale=1">  
<meta name="theme-color" content="#0f172a">  
<title>My Crypto Wallet (test mode)</title>  
<style>  
    :root { --bg:#f8fafc; --card:#ffffff; --  
text:#0f172a; --muted:#64748b; --line:#e2e8f0; --  
accent:#f7931a; --ok:#15803d; --bad:#b91c1c; }  
    @media (prefers-color-scheme: dark) {  
        :root { --bg:#0b1220; --card:#131c2e; --  
text:#e5e7eb; --muted:#94a3b8; --line:#25324a; --  
ok:#4ade80; --bad:#f87171; }  
    }  
    * { box-sizing:border-box; }  
    body { margin:0; font-family:system-ui,-apple-  
system,Segoe UI,Roboto,sans-serif; background:var(-  
-bg); color:var(--text); }  
    .wrap { max-width:480px; margin:0 auto;  
padding:16px; }  
    .banner { background:#fef3c7; color:#92400e;  
border-radius:10px; padding:8px 12px; font-  
size:13px; text-align:center; margin-bottom:14px; }  
    .card { background:var(--card); border:1px solid  
var(--line); border-radius:14px; padding:16px;  
margin-bottom:14px; }  
    h1 { font-size:20px; margin:0 0 4px; }  
    h2 { font-size:16px; margin:0 0 10px; }  
    .muted { color:var(--muted); font-size:13px; }  
    .big { font-size:28px; font-weight:700;  
margin:2px 0; }  
    .row { display:flex; gap:8px; }  
    input { width:100%; padding:12px; font-size:16px;
```

```

border-radius:10px; border:1px solid var(--line);
background:var(--bg); color:var(--text); margin-
bottom:8px; }
    button { padding:12px 14px; font-size:16px;
border:0; border-radius:10px; background:var(--
accent); color:#111; font-weight:600;
cursor:pointer; }
    button.alt { background:transparent; color:var(--
text); border:1px solid var(--line); }
    button.full { width:100%; }
    #msg { min-height:20px; font-size:14px;
margin:6px 0 12px; text-align:center; }
    .ok { color:var(--ok); } .bad { color:var(--bad);
}
    .hide { display:none; }
</style>
</head>
<body>
<div class="wrap">
    <div class="banner">TEST MODE: all money here is
fake</div>
    <div id="msg"></div>

    <div id="auth" class="card">
        <h1>My Crypto Wallet</h1>
        <p class="muted">Create a test account or log
in. Password needs 10+ characters.</p>
        <input id="email" type="email"
placeholder="Email" autocomplete="email">
        <input id="pw" type="password"
placeholder="Password" autocomplete="current-
password">
        <div class="row">
            <button class="full"
onclick="auth('signup')">Sign up</button>
            <button class="full alt"
onclick="auth('login')">Log in</button>
        </div>
    </div>
</div>

```

```

<div id="app" class="hide">
  <div class="card">
    <div class="muted">Cash balance</div>
    <div class="big" id="gbp">£0.00</div>
    <div class="muted" style="margin-
top:10px">Bitcoin balance</div>
    <div class="big" id="btc">0.00000000
BTC</div>
    <div class="muted" id="btcval"></div>
    <div class="muted" style="margin-
top:8px">Test price: 1 BTC = £50,000.00</div>
  </div>

  <div class="card">
    <h2>Add test money</h2>
    <p class="muted">Pretend bank deposit (£)</p>
    <input id="amtAdd" type="number"
inputmode="decimal" placeholder="100.00">
    <button class="full"
onclick="addMoney()">Deposit</button>
  </div>

  <div class="card">
    <h2>Buy or sell Bitcoin</h2>
    <p class="muted">Amount in £</p>
    <input id="amtTrade" type="number"
inputmode="decimal" placeholder="25.00">
    <div class="row">
      <button class="full"
onclick="trade('buy')">Buy</button>
      <button class="full alt"
onclick="trade('sell')">Sell</button>
    </div>
  </div>

  <div class="card">
    <h2>Withdraw to bank</h2>
    <p class="muted">Amount in £ (pretend payout)
</p>
    <input id="amtOut" type="number"

```

```

inputmode="decimal" placeholder="20.00">
  <button class="full"
onclick="withdraw()">Withdraw</button>
</div>

  <button class="full alt" onclick="logout()">Log
out</button>
</div>
</div>

<script>
const FAKE_PRICE_PENCE = 5000000;
let token = localStorage.getItem("token");
const $ = id => document.getElementById(id);
const gbp = p => "£" + (p /
100).toLocaleString("en-GB",
{minimumFractionDigits: 2, maximumFractionDigits:
2});
const btc = s => (s / 1e8).toFixed(8) + " BTC";

function say(text, bad) {
  const m = $("msg");
  m.textContent = text || "";
  m.className = bad ? "bad" : "ok";
}

async function api(path, method, body) {
  const headers = {"Content-Type":
"application/json"};
  if (token) headers["Authorization"] = "Bearer " +
token;
  const r = await fetch(path, {method: method ||
"GET", headers, body: body ? JSON.stringify(body) :
undefined});
  let data = {};
  try { data = await r.json(); } catch (e) {}
  if (!r.ok) {
    let d = data.detail;
    if (Array.isArray(d)) d = "Please check what
you typed (password needs 10+ characters).";

```

```

        throw new Error(d || "Something went wrong");
    }
    return data;
}

function pence(id) {
    const v = parseFloat($(id).value);
    if (!isFinite(v) || v <= 0) throw new
Error("Enter an amount above 0");
    return Math.round(v * 100);
}

async function auth(kind) {
    try {
        const d = await api("/auth/" + kind, "POST",
{email: $("email").value.trim(), password:
$("pw").value});
        token = d.token;
        localStorage.setItem("token", token);
        say("");
        await refresh();
    } catch (e) { say(e.message, true); }
}

async function refresh() {
    try {
        let d = await api("/me/balances");
        if (d.kyc_status !== "approved") {      // test
mode: identity check is simulated
            await api("/dev/approve-kyc", "POST");
            d = await api("/me/balances");
        }
        const get = a => (d.balances.find(b => b.asset
=== a) || {available: 0}).available;
        $("gbp").textContent = gbp(get("GBP"));
        $("btc").textContent = btc(get("BTC"));
        $("btcval").textContent = "worth about " +
gbp(Math.floor(get("BTC") * FAKE_PRICE_PENCE /
1e8));
        $("auth").classList.add("hide");
    }
}

```

```

        $("app").classList.remove("hide");
    } catch (e) { logout(); }
}

async function act(fn, okText) {
    try { await fn(); say(okText); await refresh(); }
    catch (e) { say(e.message, true); }
}

const addMoney = () => act(() => api("/dev/add-
test-money", "POST", {amount: pence("amtAdd"))},
"Test money added");
const trade = side => act(() => api("/trade/" +
side, "POST", {gbp_pence: pence("amtTrade"))},
side === "buy" ? "Bought
Bitcoin at the test price" : "Sold Bitcoin at the
test price");
const withdraw = () => act(() =>
api("/withdrawals", "POST",
{asset: "GBP", amount:
pence("amtOut"), destination: "test-bank-
account"}),
"Withdrawal requested
(pretend)");

function logout() {
    token = null;
    localStorage.removeItem("token");
    $("app").classList.add("hide");
    $("auth").classList.remove("hide");
}

if (token) refresh();
</script>
</body>
</html>
"""

@app.get("/", response_class=HTMLResponse)

```

```
def home():  
    return PAGE
```